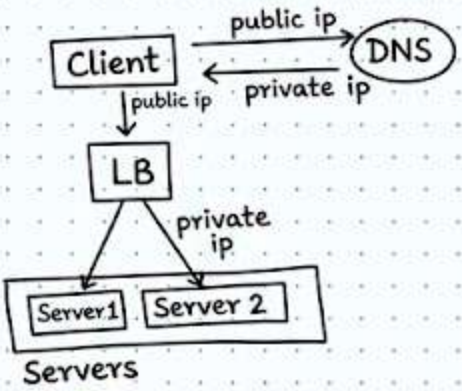
Lecture-4
Consistent Hashing || Load Balancers



Flow

Client → DNS → Load Balancer Server

- Load Balancer ka kaam hota hai - incoming requests ko multiple servers mein distribute karna to reduce traffic on single server

Master - Slave Architecture

- Replication : multiple copies of DB
  - ↳ Same copy of data from DB1 to DB2 , DB3 , etc.
- Why Replication ?
  - ↳ If main DB crash ho jaye yaa respond na kare toh -
    1. request koi replica handle karle.
    2. High availability of our DB

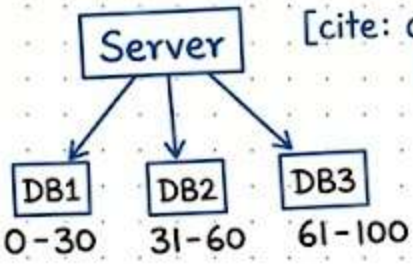
Sharding

- Why Sharding

→ Ram kabhi na kabhi full ho sakti hai , isliye sharding ka use hota hai taaki data ko chote - chote parts (shards) mein tod

Data alag-alag machines ya storage units par distribute kiya ja sake. Isse system scalable, fast aur efficient ban jaata hai.

0-100 Users



[cite: current Ram : 32 GB
[cite: ND : TB

Sharding Layers :

Data ko range based DBs mein divide karta hai

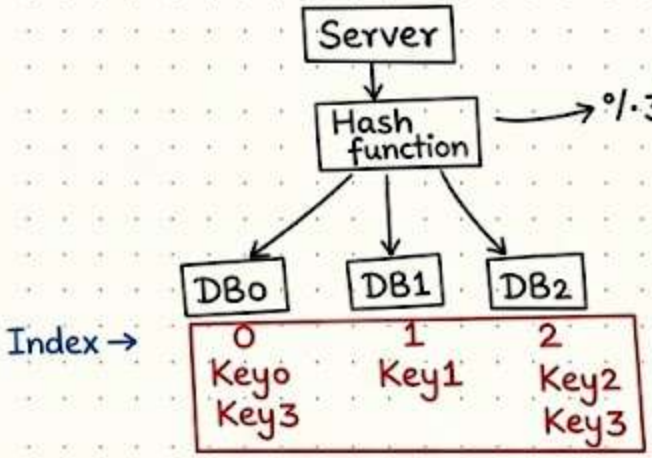
Database	user-id range
DB 1	0-30
DB 2	31-60
DB 3	61-100

- DB ka load kam hota hai
- Scalability improve hoti hai

Ex-flow
user-id = 32 → request to DB2
user-id = 15 → request to DB1

Hashing Methods : Module Operator Algorithm

1) userid	index
2) Key 0	0
Key 1	1
Key 2	2
Key 3	3



Few Examples (No. of DB = 4)

user-id	Hash (%)	DB Index
40	40% 4 = 0	DB0
3	3 % 4 = 3	DB3
12	12 % 4 = 0	DB0
43	43 % 4 = 3	DB3

No. of operation se user requests ko shard kiya jaata hai

Problems with Traditional Sharding

1) What if load increases? or Any DB gets full?
   → ek naya DB add karna padega (sol^{n}: re-sharding needed)
   → ya koi server down ho jaaye to use remove karna padega
   → Sol^{n} : AutoScaling Support hona chahiye (full db migration) (expensive operation)
2) If we increased no. of db
   → Hash function bhi badla hai
   → Full db Ko re-indexing karna padega + Migration karna padega
   → High Cost + Complexity

→ Hash func = % 3 → % 4
→ Full data redistribute karna padega
→ Re-hashing of all user IDs
→ Heavy DB Migration

3) If a DB goes down (eg: DB 2)

→ firse keys ko adjust karna padega.
→ Hash func wapas % 3 karna hoga.
→ Mapping change karni padegi.
→ Complex + too much migrations.

Conclusion

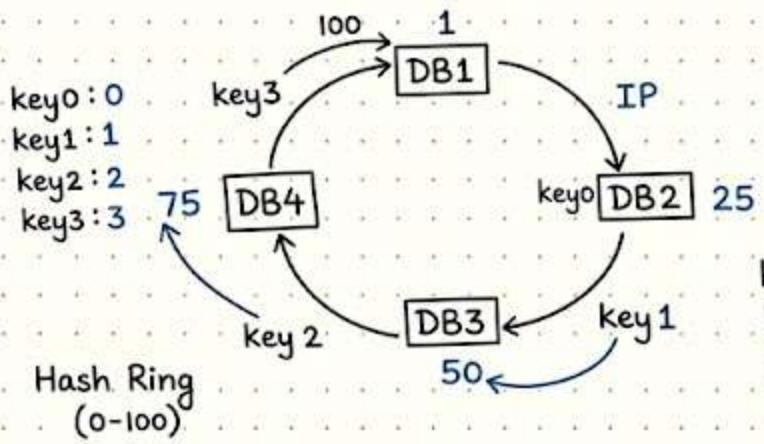
Traditional Hashing-based sharding mein Scaling aur failure handling bohot costly aur unstable ho jaata hai, isliye consistent Hashing ki need hai.

Consistent Hashing

- technique that allows minimum data migration ke saath easy scaling (add / remove DB nodes)

- **What is consistent Hashing**
  ↳ Ismein ek hash ring (hash circle) ka use karte hai... jismein keys aur database nodes dono ko circle ke points par map kiya jaata hai.
  ↳ Hash Ring ka range hota hai : 0-100.

Consistent Hashing: Hash Ring Operations



hash(IP) = 1-100
hashRing
DB hashing
key hashing

hash(key0) = 11
hash(key1) = 41
hash(key2) = 65
hash(key3) = 80

Standard deviation should be minimized

- **key Mapping Logic**

  How to store key?

  - Hash the key first → move in clockwise direction till you find your first DB Node → add the key in Node

- **Advantage -**

  1. Adding a new Node (eg. DB5)
     - Suppose hashed position of DB5 = 40
     - Ab agar hash(Key1) = 41 hai
       - ↳ toh key1, joh pehle DB3 = 50 ko point kar raha tha
       - ↳ ab clockwise direction mein DB5 (40) ko point karega
         - only key1 affected (minimum migration)
         - baki keys apni direction mein DB5 (40) jagah bani rehti hai.

Limitations of consistent Hashing

1. Uneven Distribution of keys

- jab keys ke hash values mein kaam variation hota hai ∴ sab keys ek hi db mein map ho sakti hai

  Ex: all < 25 → DB 2
  ↳ hash value

- load imbalance hoga
- ek hi db hotspot ban jaayega
- remaining dbs (DB1, DB2, DB5) idle / underutilized rehte hai

2. Celebrity problem / Hotspot key Problem

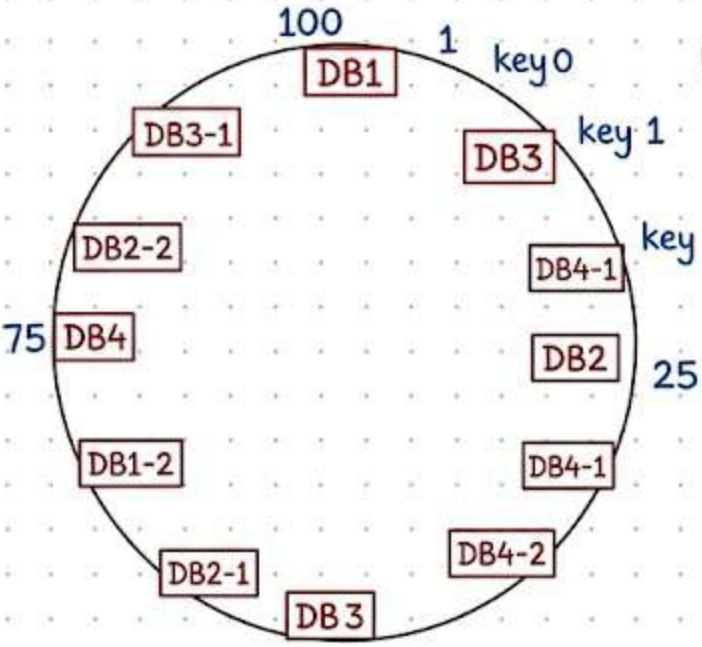
- koi ek key par jyada traffic aa raha hai
- even with consistent hashing, woh key ek hi node ko hit karega
- leading to:
  1. Single DB overload
  2. System-wide performance degrade

Solution ⇒ Virtual Nodes (vNodes)

⊙ What are virtual Nodes ?

- Har physical DB ko multiple virtual nodes / replicas ke form mein request kiya jaata hai.

  Ex: DB1 → DB 1·1 , DB 1·2
  DB2 → DB 2·1 , DB 2·2



hash(x) = ? (This should be good)

Standard deviation = High / Low

key 4 = SRK
key 5 = Virat Kohli
key 6 = Ronaldo
key 7 = Modi

How this work ?

- Yeh virtual nodes hash rung par alag-alag jagah par distribute hote hai
- Isse database, presence density badh jgati hai

Why this solution ?

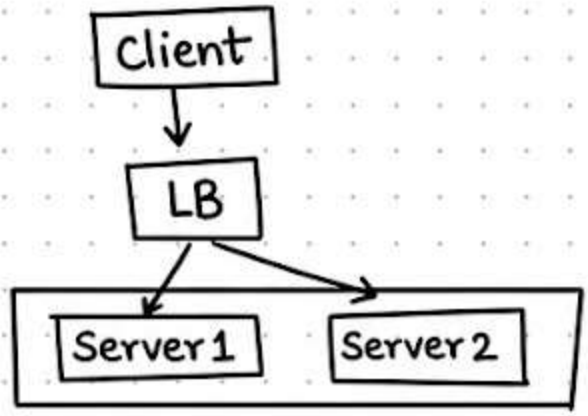
- keys ab jyada evenly distribute hote hai
- EK hi region mein multiple dbs ke virtual node ho, Sakte hai

1. Isse load spread hota hai
2. hotspot banne ka problern kaam bota hai
3. Low standard deviation issue Solve hota hai

Load Balancing

- Load balancing helps in distribute incoming traffics into multiple backend servers.
- LB targets to keep all server in balanced load so that no server gets overloaded.

- Flow



- Functions of Load Balancer

1. Which Server is available.
2. Which server can handle request.
3. And then sends the request to server.

- ensures no server is overload,
- System efficiency ↑es.
- Availability of System ↑es.

Applications of Load Balancing

1. Scalability
   - Autoscaling supported.
   - depending on traffic, server from server pool scale up/down.
   - efficient handling of fluctuating traffic.

2. Availability
   - Increase, any server fails
     1. load Balancer detects the server
     2. stops sending request to that server
   - Ensures high availability and Zero Downtime.

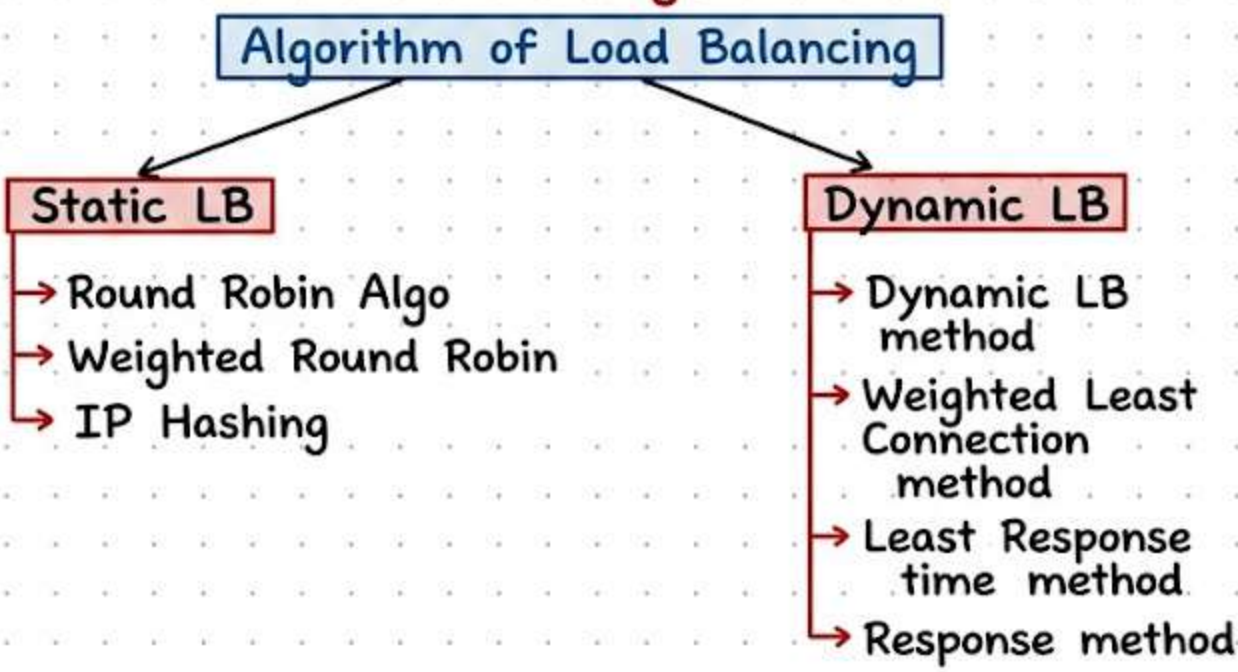
3. Security

- **DDOS Attack prevention**
  1. Malicious traffic ko filter karta hai
  2. Backend servers tak nhi pahuchne dete
- **Firewall integration**
  1. Bot, malware jaisi unwanted requests ko block karta hai

4. Responsiveness

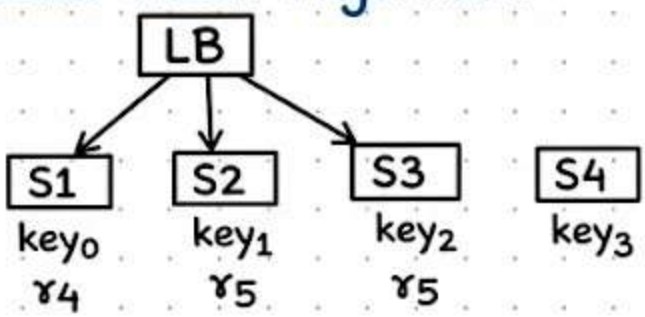
- Requests ko load multiple servers mein divide hota hai
- Isse servers fast respond karte hai
- Better user experience ensure hota hai

Algorithms of Load Balancing



Static Load Balancing Algorithm

1. Round Robin Algorithm



How it works ?

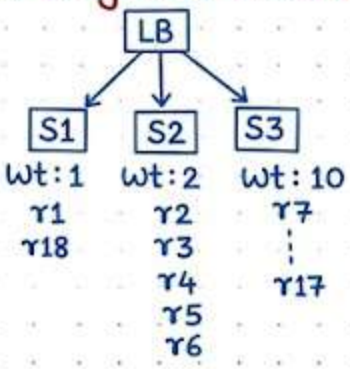
1. Sabse simple aur easy to implement algorithm.
2. Requests ko sequently servers ko assign kata hai

Req 1 ⟶ Ser 1
Req 2 ⟶ Ser 2
⋮
Req 4 ⟶ Ser 1.

Limitation

1. Server ke current load ke dhyaan mein nhi rkhta.
2. Shi Sabhi servers ko equal request deta hai, chahe woh busy ho ya free
3. good for small applications.

2. Weighted Round Robin Algorithm.



- How it works ?
- Har server ko uski capacity (Ram, CPU) ke hisab se ek weight diya jaata hai
- jyada weight wale server koi jyada requests, milti hai

Example

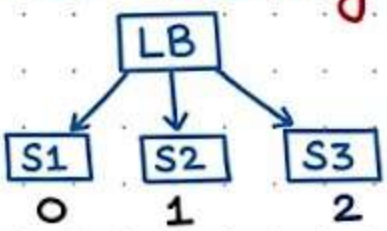
Server	Weight	Request Distribution (proportional)
S1	3	3 req
S2	5	5 req
S3	10	10 req

⊙ Total requests 18 ho toh distribution isse ratio mein hoga.

Limitations

1. Server ke [real-time load ke monitor] nhi karta
2. Weight fixed rhta hai , dynamic load variation ko handle nhi karta.

3. IP Hashing Algorithm



r1 : IP ; hash(x) = ?(MOD)
r2 : IP ; hash(x) = ?

⊙ How it works ?

→ Client ko IP address ko hash krke determine karta hai ki usse kon se server par bhejna hai.

⊙ Formula

server_index = hash (client_IP) % no. of servers

Tightly couples (stateful architecture)

↳ cox some IP ka hash same hi ayenga ⇒ so ek user ek hi server par jaayenga.

⊙ Limitation

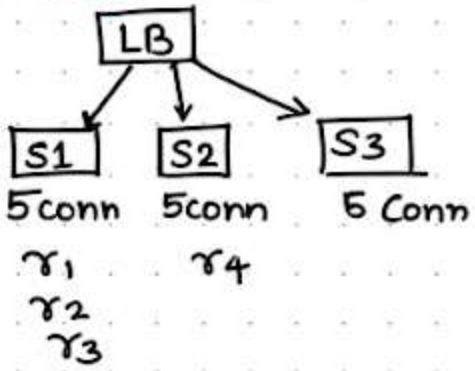
→ Hotspot problem

- Agar koi clients same public IP share kar rahe hai (Ex - office / college N/w)
- Tob sabki requests ek, hi server ho jaayengi - woh server overload ho sakta hai
- baaki server under utilized rahega

Dynamic Load Balancing

- Dynamic algorithms request distribution ka decision real-time server state ke aadhar par lete hai - jaise isi active connections, response time, yaa server health

1] Least Connection Method.

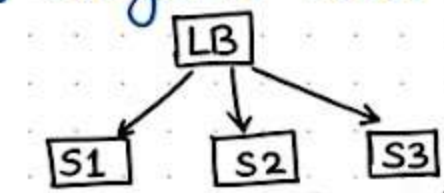


◉ How it works :
- Requests uss user server ko bheji jaati hai jiske paas sabse kaam connections hai

◉ Best suited when :
1. Sabhi connections ka size similar hoo.
2. Connections long-lived hoos.

Advantage
1. Uneven load distribution ko avoid karta hai
2. do not put much load on already busy servers

2] Weighted Least Connection Method



No. of connection / weight

◉ How it works ?
- Least connection ka enhanced version
- Har server ko ek weight assign kiya jaata hai (CPU / RAM ke hisab se)

◉ Load balancer calculate karta hai

$$\frac{\text{active - connections}}{\text{Weight}}$$

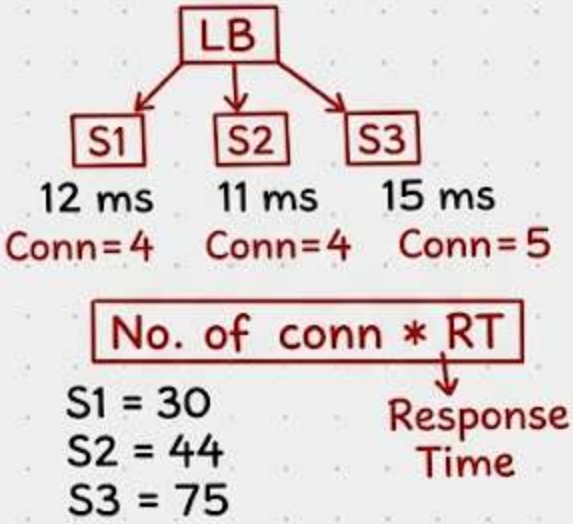
acc to this ratio ke anusar req ko assign karta hai

Example
Agar S1 ka weight 5 hai aur S2 ka 10, aur dono par 10 connections hai, toh S2 ko next request milegi coz-

S1 : 10/5 = 2.0

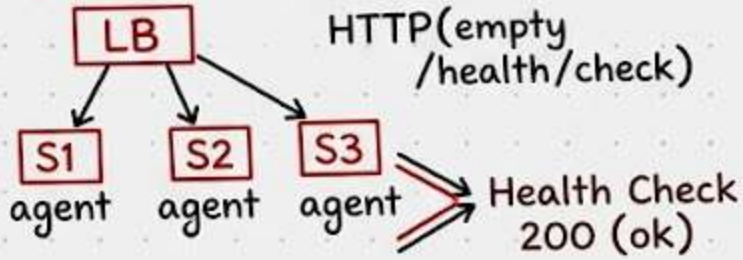
S2 : 10/10 = 1.0 → lower ratio

[3] Least Response Time Method



- How it works ?
  - Requests ko server par bhejna jaata hai, jiska response time sabse kaam hai.
  - Response time ko health check ke madat se measure kiya jaata hai.
- Useful for -
  1. High traffic websites
  2. Applications where low latency is critical

4) Resource Method



- How it works ?
  1. Load balancer har server ke live resources ko monitor karta hai
     ↳ CPU, RAM, Active connections, response time, latency.
  2. Joh server ke pass sbse kaam load hota hai, request wahi jaata hai

Capacity / load
1. no. of active conn
2. response time
3. latency
4. health check

Agent
1. Checks if server is up
2. At a particular moment what is capacity or load in that server.

Agent Involved
- Har server par ek agent hota hai
- Yeh agent real-time metrics collect karta hai aur load balancer ko bhejta hai

- Health Checks
  - Determine karta hai ki koi server "up" hai yaa "down". ⟫ UP/DOWN

How it works

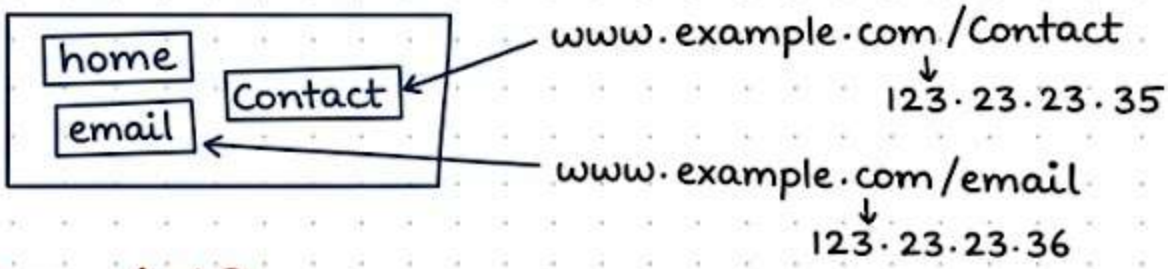
1. Load balancer server to HTTP request bheja hai (Ex : / health)
2. Agar response : HTTP 200 ok aata hai → Server
3. Agar health check fail ho joata hai → load balancer uss user ko request bhejna band kare deta hai

◉ Why Important ?

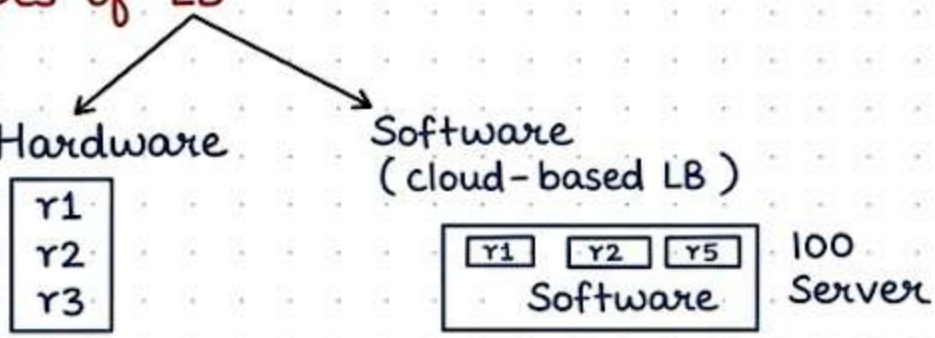
- faulty yaa dead servers avoid karne se -
  1. System ki availability bani rehti hai
  2. better experience for user

Types of load Balancer : Based on layers

1. Application level DB (HTTP/HTTPS)
2. Network layer DB (IP Address)
3. Global server load balancing (data center : GEO-DNS)
4. DNS load balancer



Types of LB



- Data center
- GB's of request [r1 , r2 , [z-l z/red , shims . r5]`

- Lower ratio = 1·1 · 200 ÷ r3 + 1/4 = 1·5 (add) r5